Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor:	M.C Rene Adrian Davila Perez
Asignatura:	Programación orientada a objetos
Grupo:	7
No de Práctica(s):	Etapa 2 Proyecto
Integrante(s):	323036285 323279866 323209432
	323161891 323063609
No. De Lista o brigada	3
Semestre:	2027-1
Fecha de entrega:	26/10/2026
Obervaciones:	

CALIFICACIÓN: _____

Índice

1. Introducción **2**

2. Marco Teórico **3**

3. Desarrollo **7**

3.1. Paquete pacientes 7
3.2. Paquete interfaces y biblioteca JAR 9
3.3. Paquete Médicos 9
3.4. Paquete Enfermeros 11
3.5. Paquete hospital.sistema y Clase Sistema 12

4. Resultados **15**

5. Conclusión **19**

1. Introducción

Actualmente los estudiantes de diferentes áreas usan los modelos de lenguaje a gran escala para consultar conceptos técnicos específicos de programación, lo que genera un problema ya que muchos usan la información sin verificarla y sin saber si es correcta, afectando la calidad y confianza del aprendizaje.

Al mismo tiempo, al diseñar un sistema más complejo en Java, como el simulador de un sistema hospitalario, el estudiantado se enfrenta al reto de organizar correctamente una jerarquía de clases con distintos niveles de especialización, como médicos generales y especialistas, enfermeros generales, especialistas y jefes de enfermería, y pacientes generales y de especialidad. Esto exige aplicar de forma conjunta el encapsulamiento, la herencia, el polimorfismo estático y dinámico, las clases abstractas y las interfaces, además de organizar y documentar correctamente el proyecto.

Resolver estos problemas nos ayudará a usar mejor a los LLM a la hora de consultar conceptos técnicos, consiguiendo una mejor comprensión y más veracidad en nuestro aprendizaje, también nos ayudará a comprender mejor el diseño orientado a objetos en Java y a practicar la organización de un proyecto más grande dentro de un package, documentado y empaquetado correctamente.

Objetivos

- Diseñar e implementar la jerarquía de clases del sistema hospitalario (médicos, enfermeros y pacientes, con sus respectivas especializaciones), para aplicar correctamente encapsulamiento, herencia y polimorfismo.
- Emplear clases abstractas e interfaces para modelar los comportamientos comunes y obligar a las clases derivadas a implementar sus propios métodos.
- Organizar el proyecto dentro de un package, generando su documentación con Javadoc y empaquetándolo en un archivo JAR, para facilitar su entendimiento y uso a diferentes usuarios.

2. Marco Teórico

Para esta segunda etapa del proyecto se emplearon los conceptos de encapsulamiento, herencia, polimorfismo estático y dinámico, entre otros conceptos fundamentales de la programación orientada a objetos [4]. A continuación se detallan los conceptos aplicados:

2.1. Encapsulamiento

El encapsulamiento sirve para proteger los datos o atributos con los métodos y para acceder a ellos se necesitan métodos del propio objeto [4].

En el diseño de clases se debe cuidar la inmutabilidad: no todos los atributos deben tener métodos `set`. Atributos como la cédula profesional, nombre o especialidad médica deben inicializarse una sola vez en el constructor y no cambiarse posteriormente para mantener la consistencia de los datos [6].

2.2. Paquetes (Packages)

Se implementó la directiva `package` para la agrupación de clases y el control de accesibilidad de clases y miembros [5]. Se utiliza el Fully Qualified Name: `nombre.subpaquete.subpaquete`.

2.3. Archivos JAR

Generamos archivos JAR; estos permiten incluir en un archivo varios archivos diferentes en un formato comprimido, pero no necesitan ser descomprimidos para usarse [4]. Todo archivo JAR contiene:

2.4. Documentación con Javadoc

Para la documentación utilizamos `javadoc`, que permite que las clases tengan un formato en HTML; para eso se necesitan generar comentarios de documentación como:

```
   /**
* comentarios de documentación
*/
```

Esta herramienta genera una descripción general del objeto de la clase, definiendo campos, constructores y métodos. Las etiquetas fundamentales empleadas conforme a los lineamientos del curso son las siguientes [4]:

- **@author:** nombre(s) y número de cuenta de los integrantes de la brigada.
- **@version:** descripción de la versión del software (versión 1.0).
- **@param:** nombre del parámetro recibido y descripción de su función.
- **@return:** descripción del valor de retorno devuelto por el método.

2.5. Herencia

El uso de la herencia nos permite crear clases a partir de clases ya existentes (de la más general a la más específica) y esto ayuda para poder reutilizar el código y usar polimorfismo [2].

- **La clase base o superclase:** es la que se usa para heredar.
- **La clase derivada o subclase:** es la que hereda de una superclase.

En la clase derivada, para acceder a los elementos de la clase base se utiliza la palabra `super`.

2.6. Sobrescritura (Overriding)

Además, usamos sobrescritura (*overriding*) para poder redefinir el comportamiento de un método acorde a las necesidades de la clase [2]. Para que el método sea sobrescrito debe tener:

- **mismo nombre.**
- **mismo tipo y número de parámetros.**
- **tipo y nivel de acceso igual o más accesible.**

- **valor de retorno del mismo tipo o un subtipo.**

2.7. Polimorfismo

El polimorfismo es una propiedad por la que es posible enviar mensajes sintácticamente iguales a objetos de tipos distintos [3].

- **Polimorfismo dinámico:** no se especifica el tipo de datos sobre el que se trabaja en tiempo de compilación; la referencia determina el método invocado, pero se ejecuta el comportamiento del objeto real instanciado.
- **Polimorfismo estático:** los tipos de datos que se pueden utilizar deben especificarse, resuelto en tiempo de compilación mediante sobrecarga de métodos (métodos con el mismo nombre y diferentes parámetros).

2.8. Clases Abstractas

En estas clases no se pueden instanciar o crear objetos; sirven para crear clases derivadas [1]. Características:

- **Métodos abstractos y concretos.**
- **Atributos con modificadores de acceso.**
- **Heredar de otras clases.**

2.9. Interfaces

Por último, para las interfaces usamos una llamada `AtencionMedica`. Una interfaz es similar a una clase abstracta pura, es decir, no es posible crear instancias de una interfaz, pero permite establecer la forma de una clase (nombres de métodos, listas de argumentos y tipo de retorno) sin bloques de código [1].

2.10. Estructuras de Datos Utilizadas

En el proyecto se utilizaron las siguientes colecciones [4]:

- **ArrayList:** estructura lineal dinámica usada para manejar las listas de médicos, enfermeros, pacientes en espera/consulta y pacientes curados.
- **HashMap:** estructura de pares clave-valor que se usó en el método `asignarPaciente()` para verificar las especialidades en minúsculas con `toLowerCase()` y `containsKey()`.
- **TreeSet:** colección ordenada basada en árbol usada en `verListaPacientes()` para ordenar alfabéticamente los apellidos de forma descendente (Z-A).

3. Desarrollo

3.1. Paquete pacientes

Para el desarrollo del programa de esta segunda etapa, el sistema hospitalario se separó en paquetes y clases siguiendo las jerarquías de herencia, encapsulamiento, clases abstractas e interfaces. A continuación se describe cómo se programó cada clase y cómo funcionan sus métodos.

Primero, se comenzó con la clase abstracta `Paciente`, ubicada en el paquete `mx.unam.die.poo.g7.etapa_2.src.pacientes`, la cual sirve como clase base para todos los pacientes del hospital. Los atributos de esta clase son:

- `nombre`: de tipo `String`, con visibilidad `private`.
- `apellidos`: de tipo `String`, con visibilidad `private`.
- `estado`: de tipo `char`, con visibilidad `private`.

Para el encapsulamiento de esta clase, los atributos `nombre` y `apellidos` no tienen métodos `set`, únicamente tienen `getNombre()` y `getApellidos()`. Esto se debe a que son los datos de identificación del paciente que se definen al momento de su registro y deben permanecer inmutables para no alterar los expedientes del hospital. Por otro lado, el atributo `estado` sí tiene el método `setEstado(char nuevoEstado)`, ya que el estado del paciente debe cambiar a lo largo del flujo hospitalario: inicia en espera (`'E'`), puede pasar a registrado (`'R'`) mediante `registroEnSistema()`, se confirma en espera (`'E'`) al solicitar consulta, avanza a consulta (`'C'`) con el médico, pasa a tratamiento (`'T'`) tras la prescripción médica y finalmente alcanza el estado de curado o alta médica (`'A'`) al recibir los cuidados de enfermería.

El constructor `Paciente(String nombre, String apellidos)` inicializa los atributos `nombre` y `apellidos`, y le asigna al atributo `estado` el valor `'E'` (En espera).

En cuanto a los métodos de la clase `Paciente`:

- `registroEnSistema()`: cambia el atributo `estado` a `'R'` e imprime un mensaje confirmando que el paciente ha sido registrado en el sistema del hospital.
- `verTratamiento()`: evalúa mediante una estructura `else if` el valor almacenado en el atributo `estado`. Si es `'E'` asigna "En espera", si es `'R'` asigna "Registrado", si es `'C'` asigna "En consulta", si es `'T'` asigna "En tratamiento" y si es `'A'` asigna "Curado", imprimiendo los datos del paciente y su fase de atención.
- `solicitarConsulta()`: método abstracto que no tiene cuerpo y termina en punto y coma, para obligar a las clases derivadas a definir cómo solicitan la consulta.
- `getTipoAtencion()`: método abstracto que retorna un `String` con el nombre de la atención requerida.

Posteriormente, se programó la clase `PacienteGeneral`, la cual hereda de `Paciente` usando la palabra `extends`. Esta clase representa a los pacientes que acuden a consulta general. Incorpora el atributo propio `motivoConsulta` de tipo `String` con visibilidad `private`, el cual sólo tiene método `getMotivoConsulta()` y no tiene `set` porque el motivo se registra al ingresar. Su constructor recibe nombre, apellidos y `motivoConsulta`; mediante `super(nombre, apellidos)` delega la inicialización a la clase base e inicializa su motivo. Sobrescribe el método `solicitarConsulta()` asegurando que el paciente se mantenga o reasigne al estado `'E'` e imprimiendo que solicita consulta de medicina general por el motivo registrado, sobrescribe `getTipoAtencion()` retornando `"Medicina General"`, y agrega el método propio `mostrarFichaGeneral()` para mostrar la información del paciente.

Después, se programó la clase `PacienteEspecialidad`, la cual también hereda de `Paciente` con `extends`. Representa a los pacientes que requieren interconsulta médica especializada. Tiene el atributo propio `especialidad` de tipo `String` con visibilidad `private` (sólo con `getEspecialidad()`, sin `set` para evitar que se desvíe a otra especialidad). Su constructor invoca a `super(nombre, apellidos)` e inicializa `especialidad`. Sobrescribe `solicitarConsulta()` manteniendo el estado `'E'` e indicando que solicita atención con la especialidad requerida, sobrescribe `getTipoAtencion()` retornando su especialidad, e incorpora el método propio `mostrarFichaEspecialidad()`.

3.2. Paquete interfaces y biblioteca JAR

Para el paquete `mx.unam.die.poo.g7.etapa_2.src.interfaces` se programó la interfaz `AtencionMedica`. Esta interfaz cuenta únicamente con el método `Paciente darTratamiento()`. Esta interfaz se empaquetó como un archivo JAR en la ruta `lib/AtencionMedica.jar`. De esta forma, las clases de personal médico y de enfermería pueden implementar la interfaz importándola como una biblioteca.

3.3. Paquete Médicos

Para el personal médico, se comenzó con la clase `Medico`, ubicada en el paquete `mx.unam.die.poo.g7.etapa_2.src.personal`, la cual implementa la interfaz `AtencionMedica`. Los atributos de esta clase son:

- `nombre`: de tipo `String`, con visibilidad `private`.
- `especialidad`: de tipo `String`, con visibilidad `private`.
- `cedula`: de tipo `int`, con visibilidad `private`.
- `noPacientes`: de tipo `int`, con visibilidad `private`.
- `listaPacientes`: de tipo `ArrayList<Paciente>`, con visibilidad `private`.

En cuanto al encapsulamiento, los atributos `nombre`, `especialidad` y `cedula` no tienen métodos `set`, únicamente tienen sus métodos `get`. La cédula profesional no debe modificarse después de crear al médico. Asimismo, `noPacientes` y `listaPacientes` no tienen `set` porque se administran internamente mediante los métodos de registro y atención para garantizar que ningún médico exceda el límite permitido de 10 pacientes simultáneos.

El constructor `Medico(String nombre, String especialidad, int cedula)` inicializa el nombre, la especialidad y la cédula, coloca `noPacientes` en 0 e inicializa la lista dinámica `listaPacientes` como un nuevo `ArrayList`.

Los métodos en `Medico` son:

- `registroEnSistema(Paciente p)`: recibe como parámetro una referencia de tipo `Paciente`. Primero valida con un `if` que `p` no sea `null` y que `noPacientes` sea menor a 10. Si se cumple, añade al paciente a `listaPacientes` e incrementa `noPacientes` en 1. Si no hay cupo, muestra un mensaje avisando que la lista está llena.
- `solicitarPaciente()`: verifica si `listaPacientes` está vacía con `.isEmpty()`. Si no está vacía, extrae al primer paciente de la lista con `pacienteTurno = listaPacientes.get(0);` si su estado no es `'C'`, invoca internamente a `darConsulta(pacienteTurno)` antes de retornarlo.
- `verListaPacientes()`: utiliza un `HashMap` y un `TreeSet<String>` para ordenar los pacientes alfabéticamente por apellido de forma descendente (Z-A), imprimiendo la lista organizada del médico.
- `darTratamiento()`: forma parte del flujo asistencial médico; toma al primer paciente, cambia su estado a `'T'` con `p.setEstado('T')`, lo retira de la lista del médico con `.remove(0)`, decrementa `noPacientes` y retorna el paciente tratado para que el hospital continúe su atención.
- `darConsulta(Paciente p)`: método abstracto que delega en las subclases cómo realiza la consulta cada tipo de médico.

Posteriormente se programó la clase `MedicoGeneral`, la cual hereda de `Medico` mediante `extends`. Representa a los médicos generales. Incorpora el atributo propio
`estudiosOrdenados` de tipo `String` con visibilidad `private`. Su constructor `MedicoGeneral(String nombre, int cedula, String estudiosOrdenados)` utiliza `super(nombre, "Medicina General", cedula)` para pasar automáticamente la especialidad base a la superclase e inicializa `estudiosOrdenados`. Sobrescribe el método `darConsulta(Paciente p)` actualizando el estado del paciente a `'C'` e imprimiendo la atención de medicina general. En la clase `Sistema`, cuando se detecta que el médico es `MedicoGeneral`, se manda a llamar a su método propio `solicitarEstudiosGenerales()`, el cual imprime la solicitud de los estudios.

Luego se programó la clase `MedicoEspecialista`, la cual hereda de `Medico`. Representa a los médicos de especialidades. Incorpora el atributo propio `pruebaEspecializada` de tipo `String` con visibilidad `private`. Su constructor `MedicoEspecialista(String nombre, String especialidad, int cedula,`

`String pruebaEspecializada)` invoca a `super(nombre, especialidad, cedula)` e inicializa `pruebaEspecializada`. Sobrescribe el método `darConsulta(Paciente p)` actualizando el estado del paciente a `'C'` e imprimiendo la atención especializada. En la clase `Sistema`, cuando el médico es `MedicoEspecialista`, se invoca su método propio `solicitarPruebaEspecializada()`, el cual imprime la solicitud de la prueba avanzada.

3.4. Paquete Enfermeros

Para el personal de enfermería, se programó la clase abstracta `Enfermero`, también en el paquete `personal` e implementando `AtencionMedica`. Sus atributos son `nombre`, `especialidad` (`String`), `cedula`, `noPacientes` (`int`) y `listaPacientes` (`ArrayList<Paciente>`), todos `private`. Al igual que con los médicos, los atributos de identificación son inmutables (sin métodos `set`) y el contador `noPacientes` está restringido a un máximo de 3 pacientes por enfermero.

Su constructor inicializa los datos y la lista de pacientes. Cuenta con el método `registroEnSistema(Paciente p)` que valida que `noPacientes` sea menor a 3 antes de ingresarlo, el método `verListaPacientes()` que muestra los pacientes organizados alfabéticamente con `TreeSet`, y el método `darTratamiento()` que coloca el estado del paciente en `'A'`, imprime su tratamiento con `verTratamiento()`, lo remueve de la lista con `.remove(0)`, decrementa `noPacientes` y retorna el paciente curado.

Para atender a los pacientes derivados de medicina general se diseñó la clase `EnfermeroGeneral`. Esta clase hereda de `Enfermero` e incorpora el atributo propio `tipoCuidado` de tipo `String` con visibilidad `private`. Su constructor delega a `super(nombre, "Medicina General", cedula)` e inicializa `tipoCuidado`. Sobrescribe el método `darTratamiento()` ejecutando su método propio `aplicarCuidadosGenerales()` y delegando el alta y salida del paciente a la superclase mediante `return super.darTratamiento();`.

Después se programó la clase `EnfermeroEspecialista`, que hereda de `Enfermero`. Incorpora el atributo propio `procedimientoAvanzado` de tipo `String` con visibilidad `private`. Su constructor invoca a `super` con su especialidad correspondiente e inicializa el procedimiento. Sobrescribe `darTratamiento()`

ejecutando su método propio `aplicarCuidadosIntensivos()` y delegando a la superclase con `return super.darTratamiento();`.

Por último, se programó la clase `JefeEnfermeria`, la cual hereda de `Enfermero`. Incorpora el atributo propio `turnoSupervision` de tipo `String` con visibilidad `private`. Su constructor recibe `nombre`, `especialidad`, `cedula` y `turnoSupervision`, lo que permite registrar jefes tanto para Medicina General como para Especialidades. Sobrescribe `darTratamiento()` indicando que supervisa la guardia y retorna `null`. Y para cumplir con el requisito de contar con una función adicional de supervisión y coordinación, se programó su método propio `supervisarGuardia(ArrayList<Enfermero>listaEnfermeros)`. Este método recibe la lista de todos los enfermeros del hospital, recorre cada enfermero con un ciclo `for` y filtra a los enfermeros generales o especialistas, imprimiendo sus nombres y mostrando el total de enfermeros asignados a esa guardia.

3.5. Paquete hospital.sistema y Clase Sistema

En el paquete `mx.unam.die.poo.g7.etapa_2.src.hospital.sistema` se programó la clase `Sistema`, la cual es la encargada de coordinar todo el hospital. Sus atributos son `nombreHospital` (`String`), los contadores `noMedicos`, `noEnfermeros`, `noPacientes` (`int`) y cuatro listas dinámicas: `listaMedicos` (`ArrayList<Medico>`), `listaEnfermeros` (`ArrayList<Enfermero>`), `listaPacientes` (`ArrayList<Paciente>`) y `listaCurados` (`ArrayList<Paciente>`). Todos los atributos son `private` y cuentan únicamente con métodos `get`.

En los métodos de registro:

- `registroMedico(Medico m)`: añade al médico recibido a la lista `listaMedicos` e incrementa `noMedicos`.
- `registroEnfermero(Enfermero e)`: añade al enfermero a `listaEnfermeros` e incrementa `noEnfermeros`.
- `registroPaciente(Paciente p)`: valida que `p` no sea `null`, añade al paciente a `listaPacientes` e incrementa el contador `noPacientes` sin

forzar arbitrariamente su estado, lo que permite registrar tanto pacientes nuevos en espera (`'E'`) como reingresar pacientes que retornan de consulta médica con estado (`'T'`).

- `agregarPacienteCurado(Paciente p)`: recibe a un paciente que ya fue tratado y lo archiva en `listaCurados`.

Para la asignación de pacientes se utilizó polimorfismo estático mediante la sobrecarga del método `asignarPaciente`:

- `asignarPaciente(Medico m)`: recorre con un ciclo `for` la lista `listaPacientes` del hospital. Para cada paciente, verifica si su estado es `'E'` y si el médico tiene cupo (`m.getNoPacientes() <10`). Para comprobar si la especialidad coincide sin problemas de mayúsculas o minúsculas, crea un `Map<String, Integer>` llamado `mapaAux` e inserta con `.put()` la (`p.getTipoAtencion().toLowerCase()`). Luego, con un
  `if mapaAux.containsKey(m.getEspecialidad().toLowerCase())`, valida si la especialidad del médico coincide. Si coincide, ingresa al paciente con `m.registroEnSistema(p)` y termina el método con `return;`.

- `asignarPaciente(Enfermero e)`: valida que el enfermero no sea `null`, que no sea una instancia de `JefeEnfermeria` y que tenga cupo disponible (`e.getNoPacientes() <3`). Recorre `listaPacientes` buscando a los pacientes que ya recibieron atención médica y tienen estado `'T'`. Si la especialidad coincide utilizando `mapaAux` con `toLowerCase()` y `containsKey()`, ingresa al paciente al enfermero con `e.registroEnSistema(p)` y finaliza con `return;`.

Por último, se programó el método `public static void main` dentro de `Sistema`, donde se ejecuta el programa completo en orden.

El programa inicia creando un objeto `Scanner` para capturar los datos desde el teclado de forma interactiva y el objeto `Sistema hospital = new Sistema()`. Se solicita al usuario el número de médicos a registrar y mediante un ciclo `for` se capturan sus datos con `Integer.parseInt(sc.nextLine())` y `sc.nextLine()`, instanciando objetos `MedicoGeneral` o `MedicoEspecialista` según la especialidad ingresada. Luego se solicita el número de enfermeros y

se registran en el hospital, agregando además a dos jefes de enfermería (uno para Medicina General y otro para Especialidad).

Inmediatamente después de registrar al personal de enfermería, se ejecuta la función de supervisión de guardia mandando a llamar a `jefe1.supervisarGuardia(hospital.getListaEnfermeros())` y `jefe2.supervisarGuardia(hospital.getListaEnfermeros())`, mostrando en pantalla los enfermeros en servicio de cada sector antes del ingreso de pacientes.

Posteriormente se piden los datos de los pacientes y se registran en el hospital con `hospital.registroPaciente(p)`. Con ciclos `for` se asignan los pacientes en espera a los médicos con `hospital.asignarPaciente(m)`. Cada médico extrae a sus pacientes con `solicitarPaciente()` y se invoca polimórficamente `darConsulta(p)`. En esta sección se empleó el operador `instanceof` como mecanismo de introspección de tipos en tiempo de ejecución (RTTI), ya que al manipular una lista polimórfica de tipo base `Medico`, `instanceof` permite verificar si el objeto concreto es `MedicoGeneral` o `MedicoEspecialista` para invocar con total seguridad sus métodos especializados propios (`solicitarEstudiosGenerales()` o `solicitarPruebaEspecializada()`), los cuales no existen en la clase base abstracta. Posteriormente, los médicos aplican `darTratamiento()` cambiando a los pacientes al estado `'T'`.

Después se realiza la asignación a enfermeros con `hospital.asignarPaciente(e)` para los pacientes en estado `'T'`. Mediante un ciclo `while` mientras el enfermero tenga pacientes (`e.getNoPacientes() >0`), se manda a llamar polimórficamente a `darTratamiento()` a través de la interfaz `AtencionMedica`. Cada paciente curado (con estado `'A'`) que retorna el enfermero es guardado en el hospital usando `hospital.agregarPacienteCurado(p)`. Finalmente, el `main` imprime el texto `"Total de pacientes curados archivados:"` junto con la cantidad obtenida de `hospital.getListaCurados().size()`.

4. Resultados

Para compilar y ejecutar el proyecto respetando la estructura de paquetes y las especificaciones de entrega, primero se compiló la interfaz `AtencionMedica` y se empaquetó en un archivo JAR en la carpeta `lib` mediante los siguientes comandos:

```
javac -d test src/interfaces/AtencionMedica.java

jar cvf lib/AtencionMedica.jar -C test \
mx/unam/die/poo/g7/etapa_2/src/interfaces/AtencionMedica.class
```

Posteriormente se compilaron todas las clases del proyecto vinculando la biblioteca generada y enviando los archivos `.class` directamente a la carpeta `test`:

```
javac -cp "lib/AtencionMedica.jar;test" -d test \
src/pacientes/*.java \
src/personal/*.java \
src/hospital/sistema/*.java
```

Y para generar el archivo JAR ejecutable autónomo y la documentación con `Javadoc`:

```
jar cfe etapa_2.jar \
mx.unam.die.poo.g7.etapa_2.src.hospital.sistema.Sistema -C test .

javadoc -d doc -cp "lib/AtencionMedica.jar" \
-subpackages mx.unam.die.poo.g7.etapa_2.src
```

Se inició con el registro interactivo del personal médico y de enfermería en el hospital a través de la consola:

```
Registro de médicos:
¿Cuántos médicos vas a registrar?: 2

Datos del Médico 1:
Tipo (1. General / 2. Especialista): 1
Nombre: Juan
Cédula: 332324
Estudios ordenados: Quimica sanguinea

Datos del Médico 2:
Tipo (1. General / 2. Especialista): 2
Nombre: Elena
Cédula: 2324242
Especialidad: Pediatria
Prueba especializada: Electrocardiograma pediatrico

Registro de enfermeros:
¿Cuántos enfermeros vas a registrar? 2

Datos del Enfermero 1:
Tipo (1. General / 2. Especialista): 1
Nombre: Alberto
Cédula: 4232
Tipo de cuidado: Curacion heridas

Datos del Enfermero 2:
Tipo (1. General / 2. Especialista): 2
Nombre: Pedro
Cédula: 443434
Especialidad: Pediatria
Procedimiento avanzado: Monitoreo continuo

Registro de Jefes de Enfermería (2 enfermeros):
1. Jefe de Enfermeros Generales:
Nombre: Brenda
Cédula: 43434
Turno de supervisión: Matutino

2. Jefe de Enfermeros Especialistas:
Nombre: Roberto
Cédula: 4343533
Turno de supervisión: Vespertino
```

A continuación se ejecutó la supervisión de guardias por parte de los jefes de enfermería, auditando a los enfermeros asignados a cada sector:

```
Supervisión de guardias de enfermería:

Supervisión de enfermería por el jefe Brenda (Medicina General) en turno Matutino:
- Enfermero General: Alberto

Supervisión de enfermería por el jefe Roberto (Especialidad) en turno Vespertino:
- Enfermero Especialista: Pedro (Pediatria)
```

Posteriormente se registraron los pacientes y se realizó la asignación médica de acuerdo con la especialidad requerida. Cada médico atendió a sus pacientes llamando polimórficamente a `darConsulta()` y solicitando los estudios correspondientes:

```
Registro de pacientes:
¿Cuántos pacientes vas a registrar? 2

Datos del Paciente 1:
Tipo (1. General / 2. Especialidad): 1
Nombre: Ana
Apellidos: Lopez Martinez
Motivo de consulta: Fiebre y cuerpo cortado
Ficha del paciente: Ana Lopez Martinez motivo: Fiebre y cuerpo cortado
Paciente Ana Lopez Martinez registrado en el sistema.
El paciente general Ana solicita atencion en medicina general por: Fiebre y cuerpo cortado

Datos del Paciente 2:
Tipo (1. General / 2. Especialidad): 2
Nombre: Diego
Apellidos: Vera Martinez
Especialidad que requiere: Pediatria
Ficha del paciente: Diego Vera Martinez especialidad: Pediatria
Paciente Diego Vera Martinez registrado en el sistema.
El paciente de especialidad Diego solicita atencion en: Pediatria
```

```
----------------------------
Hospital: Hospital La Raza

Asignación a médicos:
Paciente Ana asignado al médico Juan
Paciente Diego asignado al médico Elena

Asignación médica:

Lista de pacientes ordenada del medico :Juan
Lopez Martinez Ana - Especialidad: Medicina General (0)
El médico general Juan atiende al paciente: Ana
Paciente: Ana Lopez Martinez estado: En consulta
El médico general Juan solicita los estudios: Quimica sanguinea
El medico indico tratamiento a: Ana

Lista de pacientes ordenada del medico :Elena
Vera Martinez Diego - Especialidad: Pediatria (0)
El médico especialista en Pediatria (Elena) atiende al paciente: Diego
Paciente: Diego Vera Martinez estado: En consulta
El médico especialista en Pediatria (Elena) solicita la prueba: Electrocardiograma pediatrico
El medico indico tratamiento a: Diego
```

Posteriormente se llevó a cabo la asignación del personal de enfermería para los pacientes que requerían tratamiento. Cada enfermero ejecutó el método `darTratamiento()` mediante la interfaz `AtencionMedica`, aplicando los cuidados y dando de alta al paciente:

```
Asignación a enfermeros:
Paciente Ana asignado al enfermero Alberto
Paciente Diego asignado al enfermero Pedro

Tratamiento de enfermeros:

Lista de pacientes del enfermero :Alberto
Lopez Martinez Ana - Especialidad: Medicina General (0)
El enfermero general Alberto aplica cuidados: Curacion heridas
Dando tratamiento a: Ana
Paciente: Ana Lopez Martinez estado: Curado
Paciente Ana ahora está curado

Lista de pacientes del enfermero :Pedro
Vera Martinez Diego - Especialidad: Pediatria (0)
El enfermero especialista en Pediatria (Pedro) realiza el procedimiento: Monitoreo continuo
Dando tratamiento a: Diego
Paciente: Diego Vera Martinez estado: Curado
Paciente Diego ahora está curado
```

Finalmente se obtuvo en la consola el total de pacientes curados que fueron archivados en el hospital tras concluir su atención médica:

```
Total de pacientes curados archivados: 2
```

5. Conclusión

En esta segunda etapa del proyecto se logró hacer evolucionar el funcionamiento del sistema hospitalario mediante la aplicación de encapsulamiento, paquetes, herencia, clases abstractas, interfaces y polimorfismo, cumpliendo con todos los requisitos establecidos en la especificación.

Se implementaron las jerarquías de clases a partir de las clases base abstractas Paciente, Médico y Enfermero, cada una con sus atributos privados y sus comportamientos correspondientes. Esto permitió simular de forma más realista el registro y la atención médica y de enfermería, dividiendo a los médicos y pacientes en generales y especialistas, y añadiendo a la jefa de enfermería con su función de supervisión de guardia, conservando las restricciones de cupo (10 pacientes por médico y 3 por enfermero).

Se aplicó un encapsulamiento estricto en todos los atributos mediante el modificador private, justificando qué datos deben ser inmutables como las cédulas profesionales, nombres y especialidades, y dejando únicamente el método set para el atributo estado del paciente para controlar en qué fase se encuentra (espera, consulta, tratamiento y curado). Asimismo, se organizó el proyecto en paquetes formales y se implementó la interfaz AtencionMedica empaquetada en un archivo JAR dentro de la carpeta lib.

El uso de polimorfismo dinámico facilitó el manejo de listas polimórficas de médicos, enfermeros y pacientes, permitiendo que al invocar los métodos darConsulta y darTratamiento se ejecute automáticamente el comportamiento especializado de cada subclase en tiempo de ejecución. Con esta etapa se consolidó una estructura modular y reutilizable, dejando preparado el sistema para futuras etapas del proyecto.

Referencias

[1] GeeksforGeeks. *Abstract Classes and Interfaces in Java*. https://www.geeksforgeeks.org/difference-between-abstract-class-and-interface-in-java/. Accedido: 22 de octubre de 2026.

[2] GeeksforGeeks. *Inheritance in Java*. https://www.geeksforgeeks.org/inheritance-in-java/. Accedido: 21 de octubre de 2026.

[3] GeeksforGeeks. *Polymorphism in Java*. https://www.geeksforgeeks.org/polymorphism-in-java/. Accedido: 22 de octubre de 2026.

[4] M. A. Nakayama et al. *Manual de prácticas del laboratorio de programación orientada a objetos*. Facultad de Ingeniería, Universidad Nacional Autónoma de México. 2017. Código MADO-22, versión 01.

[5] Oracle. *Java Documentation - Classes, Objects and Packages*. https://docs.oracle.com/javase/tutorial/java/concepts/. Accedido: 21 de octubre de 2026.

[6] Portal Académico CCH-UNAM. *Constructores - Principios de Programación Orientada a Objetos*. https://portalacademico.cch.unam.mx/cibernetica2/principios-programacion-orientada-a-objetos/constructores. Accedido: 20 de octubre de 2026.